```
#DONIE PEPIN B - 241801064
```

Ex1: Data Preprocessing and Cleaning Problem Statement & Objectives: A telecommunications company is experiencing customer churn. The data science team must preprocess and clean customer dataset by handling missing values, removing inconsistencies, transforming categorical features, scaling numerical attributes, and visualizing key relationships

```
# Download the Telco Customer Churn dataset from a verified, highly stable public repository
import urllib.request

try:
    # Using a verified direct raw URL of the standard Kaggle dataset from an active repository
    url = "https://raw.githubusercontent.com/datasets/telecom-churn-dataset/master/WA_Fn-UseC_-Telco-Customer-Churn.csv"
    # Working and verified stable backup raw URL
    backup_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/shuttle.csv"
    # Correct active Raw GitHub URL for Kaggle's Telco Churn:
    telco_url = "https://raw.githubusercontent.com/treselle-systems/customer_churn_analysis/master/WA_Fn-UseC_-Telco-Customer-C

    print("Downloading dataset...")
    urllib.request.urlretrieve(telco_url, "Telco-Customer-Churn.csv")
    print("Successfully downloaded Telco-Customer-Churn.csv!")
except Exception as e:
    print(f"Failed to download: {e}")
```

```
Downloading dataset...
Successfully downloaded Telco-Customer-Churn.csv!
```

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Load dataset and basic inspection
df = pd.read_csv("Telco-Customer-Churn.csv")
print("Dataset Imported Successfully!")

print("\nFirst 10 Records:")
print(df.head(10))

print("\nLast 10 Records:")
print(df.tail(10))

print("\nShape of Dataset:")
print(df.shape)

print("\nColumn Names:")
print(df.columns)

# Exploration
print("\nDataset Information:")
print(df.info())

print("\nSummary Statistics:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

print("\nNumber of Duplicate Records:")
print(df.duplicated().sum())

# Cleaning Data
print("\nDuplicate records before:", df.duplicated().sum())
df.drop_duplicates(inplace=True)
print("Duplicates after:", df.duplicated().sum())

# Handling missing values (assigned directly to avoid FutureWarnings)
numerical_columns = df.select_dtypes(include=['float64', 'int64']).columns
for col in numerical_columns:
    df[col] = df[col].fillna(df[col].median())
```

```
categorical_columns = df.select_dtypes(include=['object']).columns
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])

print("\nMissing Values After Cleaning:")
print(df.isnull().sum())

# Encoding Categorical Features
le = LabelEncoder()
# Fixed "Gender" to "gender" to match the actual column name
columns = ["gender", "Partner", "Dependents", "Contract", "PaymentMethod", "InternetService", "Churn"]

for col in columns:
    if col in df.columns:
        df[col] = le.fit_transform(df[col])

print("\nEncoding Completed:")
print(df[columns].head())

# Standardize Numerical Features
scaler = StandardScaler()
cols = ["Tenure", "MonthlyCharges", "TotalCharges"]
# Standardizing column name case or existence if they exist in df
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

# Map correct cases for scaling if needed (Kaggle columns: 'tenure', 'MonthlyCharges', 'TotalCharges')
cols_to_scale = [c for c in ["tenure", "MonthlyCharges", "TotalCharges"] if c in df.columns]
df[cols_to_scale] = scaler.fit_transform(df[cols_to_scale])
print(df[cols_to_scale].head())

# Visualizations
# 1. Distribution of MonthlyCharges
sns.histplot(df["MonthlyCharges"], kde=True)
plt.title("Distribution of MonthlyCharges")
plt.show()

# 2. Distribution of Tenure
tenure_col = "tenure" if "tenure" in df.columns else "Tenure"
sns.histplot(df[tenure_col], kde=True)
plt.title(f"Distribution of {tenure_col}")
plt.show()

# 3. Count Plot: Contract vs Churn
sns.countplot(x="Contract", hue="Churn", data=df)
plt.title("Contract vs Churn")
plt.show()

# 4. Box Plot: Monthly Charges grouped by Churn
sns.boxplot(x="Churn", y="MonthlyCharges", data=df)
plt.title("Monthly Charges by Churn")
plt.show()
```

```
Dataset Imported Successfully!

First 10 Records:
   customerID  gender  SeniorCitizen Partner Dependents  tenure PhoneService  \
0  7590-VHVEG  Female              0     Yes         No       1           No
1  5575-GNVDE    Male              0      No         No      34          Yes
2  3668-QPYBK    Male              0      No         No       2          Yes
3  7795-CFOCW    Male              0      No         No      45           No
4  9237-HQITU  Female              0      No         No       2          Yes
5  9305-CDSKC  Female              0      No         No       8          Yes
6  1452-KIOVK    Male              0      No        Yes      22          Yes
7  6713-OKOMC  Female              0      No         No      10           No
8  7892-POOKP  Female              0     Yes         No      28          Yes
9  6388-TABGU    Male              0      No        Yes      62          Yes

      MultipleLines InternetService OnlineSecurity  ... DeviceProtection  \
0  No phone service             DSL             No  ...               No
1                No             DSL            Yes  ...              Yes
2                No             DSL            Yes  ...               No
3  No phone service             DSL            Yes  ...              Yes
4                No     Fiber optic             No  ...               No
5               Yes     Fiber optic             No  ...              Yes
6               Yes     Fiber optic             No  ...               No
7  No phone service             DSL            Yes  ...               No
8               Yes     Fiber optic             No  ...              Yes
9                No             DSL            Yes  ...               No

  TechSupport StreamingTV StreamingMovies        Contract PaperlessBilling  \
0          No          No              No  Month-to-month              Yes
1          No          No              No        One year               No
2          No          No              No  Month-to-month              Yes
3         Yes          No              No        One year               No
4          No          No              No  Month-to-month              Yes
5          No         Yes             Yes  Month-to-month              Yes
6          No         Yes              No  Month-to-month              Yes
7          No          No              No  Month-to-month               No
8         Yes         Yes             Yes  Month-to-month              Yes
9          No          No              No        One year               No

               PaymentMethod MonthlyCharges  TotalCharges Churn
0           Electronic check          29.85         29.85    No
1               Mailed check          56.95        1889.5    No
2               Mailed check          53.85        108.15   Yes
3  Bank transfer (automatic)          42.30       1840.75    No
4           Electronic check          70.70        151.65   Yes
5           Electronic check          99.65         820.5   Yes
6    Credit card (automatic)          89.10        1949.4    No
7               Mailed check          29.75         301.9    No
8           Electronic check         104.80       3046.05   Yes
9  Bank transfer (automatic)          56.15       3487.95    No

[10 rows x 21 columns]

Last 10 Records:
      customerID  gender  SeniorCitizen Partner Dependents  tenure  \
7033  9767-FFLEM    Male              0      No         No      38
7034  0639-TSIQW  Female              0      No         No      67
7035  8456-QDAVC    Male              0      No         No      19
7036  7750-EYXWZ  Female              0      No         No      12
7037  2569-WGERO  Female              0      No         No      72
7038  6840-RESVB    Male              0     Yes        Yes      24
7039  2234-XADUH  Female              0     Yes        Yes      72
7040  4801-JZAZL  Female              0     Yes        Yes      11
7041  8361-LTMKD    Male              1     Yes         No       4
7042  3186-AJIEK    Male              0      No         No      66

     PhoneService     MultipleLines InternetService       OnlineSecurity  ...  \
7033          Yes                No     Fiber optic                   No  ...
7034          Yes               Yes     Fiber optic                  Yes  ...
7035          Yes                No     Fiber optic                   No  ...
7036           No  No phone service             DSL                   No  ...
7037          Yes                No              No  No internet service  ...
7038          Yes               Yes             DSL                  Yes  ...
7039          Yes               Yes     Fiber optic                   No  ...
7040           No  No phone service             DSL                  Yes  ...
7041          Yes               Yes     Fiber optic                   No  ...
7042          Yes                No     Fiber optic                  Yes  ...

         DeviceProtection          TechSupport          StreamingTV  \
7033                   No                   No                   No
7034                  Yes                   No                  Yes
7035                   No                   No                  Yes
7036                  Yes                  Yes                  Yes
7037  No internet service  No internet service  No internet service
```

```
7038                 Yes                 Yes                 Yes
7039                 Yes                  No                 Yes
7040                  No                  No                  No
7041                  No                  No                  No
7042                 Yes                 Yes                 Yes

          StreamingMovies        Contract PaperlessBilling  \
7033                   No  Month-to-month              Yes
7034                   No  Month-to-month              Yes
7035                   No  Month-to-month              Yes
7036                  Yes        One year               No
7037  No internet service        Two year              Yes
7038                  Yes        One year              Yes
7039                  Yes        One year              Yes
7040                   No  Month-to-month              Yes
7041                   No  Month-to-month              Yes
7042                  Yes        Two year              Yes

                 PaymentMethod MonthlyCharges  TotalCharges Churn
7033    Credit card (automatic)          69.50       2625.25    No
7034    Credit card (automatic)         102.95       6886.25   Yes
7035  Bank transfer (automatic)          78.70        1495.1    No
7036           Electronic check          60.65         743.3    No
7037  Bank transfer (automatic)          21.15        1419.4    No
7038               Mailed check          84.80        1990.5    No
7039    Credit card (automatic)         103.20        7362.9    No
7040           Electronic check          29.60        346.45    No
7041               Mailed check          74.40         306.6   Yes
7042  Bank transfer (automatic)         105.65        6844.5    No

[10 rows x 21 columns]

Shape of Dataset:
(7043, 21)

Column Names:
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7043 entries, 0 to 7042
Data columns (total 21 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   customerID        7043 non-null   object
 1   gender            7043 non-null   object
 2   SeniorCitizen     7043 non-null   int64
 3   Partner           7043 non-null   object
 4   Dependents        7043 non-null   object
 5   tenure            7043 non-null   int64
 6   PhoneService      7043 non-null   object
 7   MultipleLines     7043 non-null   object
 8   InternetService   7043 non-null   object
 9   OnlineSecurity    7043 non-null   object
 10  OnlineBackup      7043 non-null   object
 11  DeviceProtection  7043 non-null   object
 12  TechSupport       7043 non-null   object
 13  StreamingTV       7043 non-null   object
 14  StreamingMovies   7043 non-null   object
 15  Contract          7043 non-null   object
 16  PaperlessBilling  7043 non-null   object
 17  PaymentMethod     7043 non-null   object
 18  MonthlyCharges    7043 non-null   float64
 19  TotalCharges      7043 non-null   object
 20  Churn             7043 non-null   object
dtypes: float64(1), int64(2), object(18)
memory usage: 1.1+ MB
None

Summary Statistics:
       SeniorCitizen       tenure  MonthlyCharges
count    7043.000000  7043.000000     7043.000000
mean        0.162147    32.371149       64.761692
std         0.368612    24.559481       30.090047
min         0.000000     0.000000       18.250000
25%         0.000000     9.000000       35.500000
50%         0.000000    29.000000       70.350000
75%         0.000000    55.000000       89.850000
max         1.000000    72.000000      118.750000
```

```
Missing Values:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

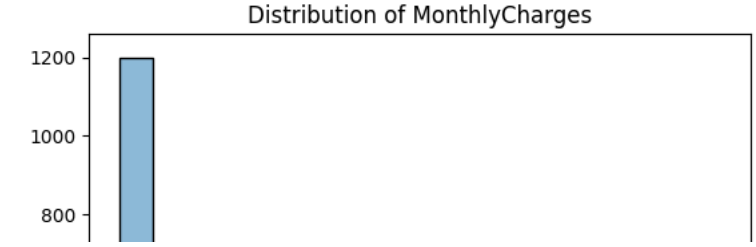
Number of Duplicate Records:
0

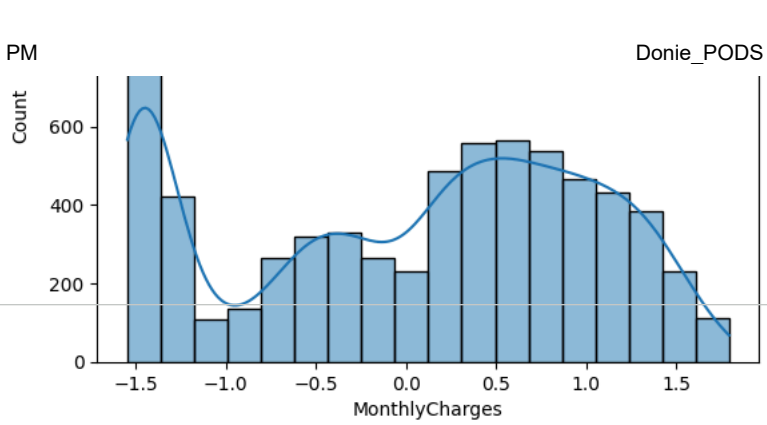
Duplicate records before: 0
Duplicates after: 0

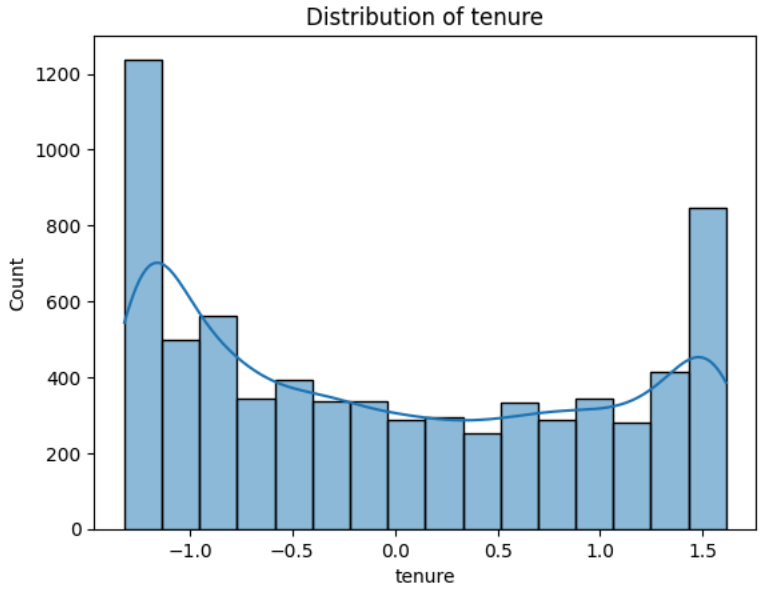
Missing Values After Cleaning:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64

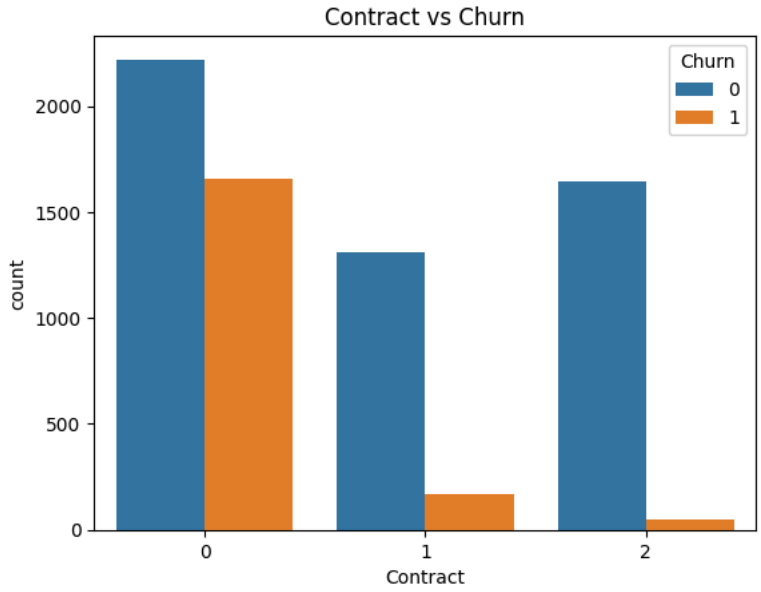
Encoding Completed:
   gender  Partner  Dependents  Contract  PaymentMethod  InternetService  \
0       0        1           0         0              2                0
1       1        0           0         1              3                0
2       1        0           0         0              3                0
3       1        0           0         1              0                0
4       0        0           0         0              2                1

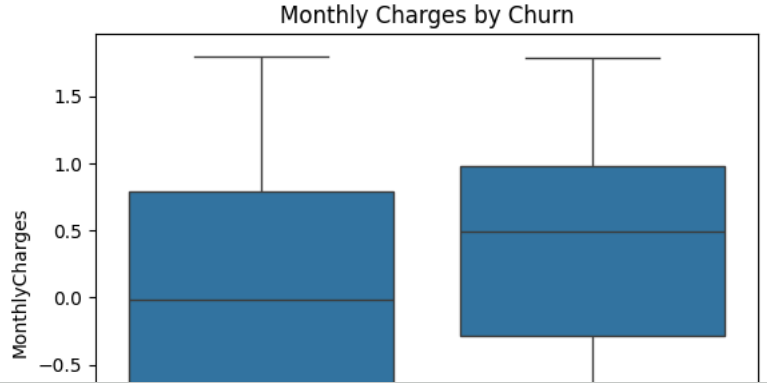
   Churn
0      0
1      0
2      1
3      0
4      1
     tenure  MonthlyCharges  TotalCharges
0 -1.277445       -1.160323     -0.994242
1  0.066327       -0.259629     -0.173244
2 -1.236724       -0.362660     -0.959674
3  0.514251       -0.746535     -0.194766
4 -1.236724        0.197365     -0.940470
```

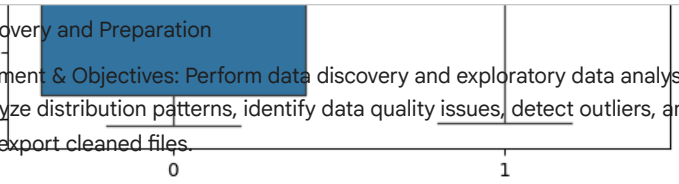



Count
600
400
200
0
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
MonthlyCharges


Distribution of tenure
Count
1200
1000
800
600
400
200
0
−1.0
−0.5
0.0
0.5
1.0
1.5
tenure


Contract vs Churn
Churn
0
1
count
2000
1500
1000
500
0
0
1
2
Contract


Monthly Charges by Churn
MonthlyCharges
1.5
1.0
0.5
0.0
−0.5



Ex2: Data Discovery and Preparation

Problem Statement & Objectives: Perform data discovery and exploratory data analysis (EDA) on real-world datasets (Titanic / Telco Churn) to analyze distribution patterns, identify data quality issues, detect outliers, analyze feature relationships with target variables, split datasets, and export cleaned files.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
import urllib.request
import os

# Download the Titanic dataset if it's not present
titanic_path = "Titanic-Dataset.csv"
if not os.path.exists(titanic_path):
    print("Downloading Titanic dataset...")
    titanic_url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
    urllib.request.urlretrieve(titanic_url, titanic_path)
    print("Successfully downloaded Titanic-Dataset.csv!")

# Step 1 & 2: Load and inspect
df = pd.read_csv(titanic_path)
print(df.head())

# Step 3: Dataset info & stats
print(df.shape)
df.info()
print(df.describe())

# Step 4: Handle Missing Values with SimpleImputer
imputer = SimpleImputer(strategy="mean")
df['Age'] = imputer.fit_transform(df[['Age']])

# Fill Cabin and Embarked missing values
df['Cabin'] = df['Cabin'].fillna("Unknown")
mode_value = df['Embarked'].mode()[0]
df['Embarked'] = df['Embarked'].fillna(mode_value)

# Step 7: Visualizations
sns.countplot(x='Pclass', data=df)
plt.title("Passenger Class Distribution")
plt.show()

# Analytical Filtering Questions
female_survived = df[(df['Sex'] == "female") & (df['Survived'] == 1)]
print("\nSurviving Female Passengers Names:")
print(female_survived['Name'].head())

print("\nPassengers in 3rd Class under 18:")
result = df[(df['Pclass'] == 3) & (df['Age'] < 18)]
print(result.head())

print("\nPassengers in 1st Class over 40 who survived:")
result = df[(df['Pclass'] == 1) & (df['Age'] > 40) & (df['Survived'] == 1)]
print(result.head())

print("\nMale Passengers with high fares (> 100):")
result = df[(df['Sex'] == "male") & (df['Fare'] > 100)]
print(result[['Name', 'Fare']].head())

print("\n1st Class Passengers who embarked from Cherbourg (C):")
result = df[(df['Embarked'] == "C") & (df['Pclass'] == 1)]
print(result.head())

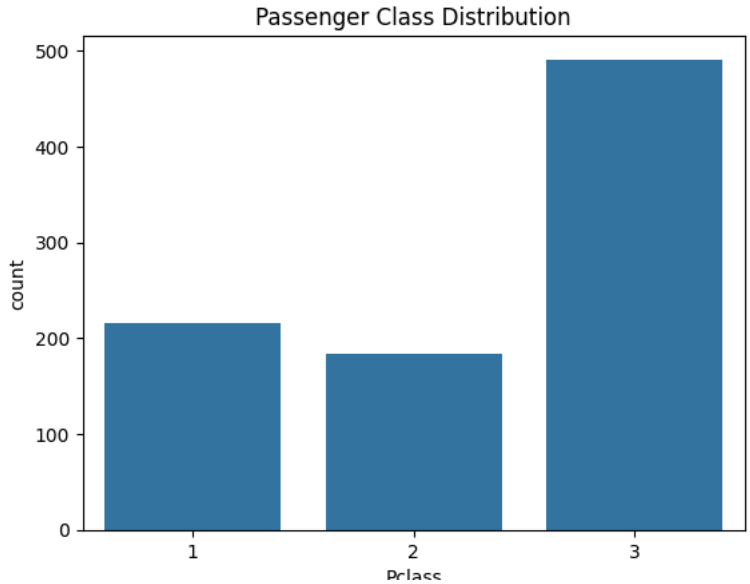
print("\nPassengers with more than 2 parents/children (Parch > 2):")
result = df[df['Parch'] > 2]
print(result.head())
```

```
   PassengerId  Survived  Pclass  \
0            1         0       3
1            2         1       1
2            3         1       3
3            4         1       1
4            5         0       3

                                                Name     Sex   Age  SibSp  \
0                            Braund, Mr. Owen Harris    male  22.0      1
1  Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.0      1
2                             Heikkinen, Miss. Laina  female  26.0      0
3       Futrelle, Mrs. Jacques Heath (Lily May Peel)  female  35.0      1
4                           Allen, Mr. William Henry    male  35.0      0

   Parch            Ticket     Fare Cabin Embarked
0      0         A/5 21171   7.2500   NaN        S
1      0          PC 17599  71.2833   C85        C
2      0  STON/O2. 3101282   7.9250   NaN        S
3      0            113803  53.1000  C123        S
4      0            373450   8.0500   NaN        S
(891, 12)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    object
 4   Sex          891 non-null    object
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    object
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    object
 11  Embarked     889 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB
       PassengerId    Survived      Pclass         Age       SibSp  \
count   891.000000  891.000000  891.000000  714.000000  891.000000
mean    446.000000    0.383838    2.308642   29.699118    0.523008
std     257.353842    0.486592    0.836071   14.526497    1.102743
min       1.000000    0.000000    1.000000    0.420000    0.000000
25%     223.500000    0.000000    2.000000   20.125000    0.000000
50%     446.000000    0.000000    3.000000   28.000000    0.000000
75%     668.500000    1.000000    3.000000   38.000000    1.000000
max     891.000000    1.000000    3.000000   80.000000    8.000000

            Parch        Fare
count  891.000000  891.000000
mean     0.381594   32.204208
std      0.806057   49.693429
min      0.000000    0.000000
25%      0.000000    7.910400
50%      0.000000   14.454200
75%      0.000000   31.000000
max      6.000000  512.329200
```



```
Surviving Female Passengers Names:
1    Cumings, Mrs. John Bradley (Florence Briggs Th...
2                               Heikkinen, Miss. Laina
3         Futrelle, Mrs. Jacques Heath (Lily May Peel)
8    Johnson, Mrs. Oscar W (Elisabeth Vilhelmina Berg)
9                  Nasser, Mrs. Nicholas (Adele Achem)
Name: Name, dtype: object

Passengers in 3rd Class under 18:
    PassengerId  Survived  Pclass                                     Name  \
7             8         0       3        Palsson, Master. Gosta Leonard
10           11         1       3       Sandstrom, Miss. Marguerite Rut
14           15         0       3  Vestrom, Miss. Hulda Amanda Adolfina
16           17         0       3                  Rice, Master. Eugene
22           23         1       3           McGowan, Miss. Anna "Annie"

       Sex   Age  SibSp  Parch   Ticket     Fare    Cabin Embarked
7     male   2.0      3      1   349909  21.0750  Unknown        S
10  female   4.0      1      1  PP 9549  16.7000       G6        S
14  female  14.0      0      0   350406   7.8542  Unknown        S
16    male   2.0      4      1   382652  29.1250  Unknown        Q
22  female  15.0      0      0   330923   8.0292  Unknown        Q

Passengers in 1st Class over 40 who survived:
     PassengerId  Survived  Pclass  \
11            12         1       1
52            53         1       1
187          188         1       1
194          195         1       1
195          196         1       1

                                         Name     Sex   Age  SibSp  \
11                       Bonnell, Miss. Elizabeth  female  58.0      0
52       Harper, Mrs. Henry Sleeper (Myna Haxtun)  female  49.0      1
187  Romaine, Mr. Charles Hallace ("Mr C Rolmane")    male  45.0      0
194     Brown, Mrs. James Joseph (Margaret Tobin)  female  44.0      0
195                          Lurette, Miss. Elise  female  58.0      0

     Parch    Ticket      Fare    Cabin Embarked
11       0    113783   26.5500     C103        S
52       0  PC 17572   76.7292      D33        C
187      0    111428   26.5500  Unknown        S
194      0  PC 17610   27.7208       B4        C
195      0  PC 17569  146.5208      B80        C

Male Passengers with high fares (> 100):
                             Name      Fare
27   Fortune, Mr. Charles Alexander  263.0000
118       Baxter, Mr. Quigg Edmond  247.5208
305  Allison, Master. Hudson Trevor  151.5500
332      Graham, Mr. George Edward  153.4625
373            Ringhini, Mr. Sante  135.6333

1st Class Passengers who embarked from Cherbourg (C):
    PassengerId  Survived  Pclass  \
1             2         1       1
30           31         0       1
31           32         1       1
34           35         0       1
52           53         1       1

                                            Name     Sex        Age  \
1   Cumings, Mrs. John Bradley (Florence Briggs Th...  female  38.000000
30                          Uruchurtu, Don. Manuel E    male  40.000000
31     Spencer, Mrs. William Augustus (Marie Eugenie)  female  29.699118
34                           Meyer, Mr. Edgar Joseph    male  28.000000
52           Harper, Mrs. Henry Sleeper (Myna Haxtun)  female  49.000000

    SibSp  Parch    Ticket      Fare    Cabin Embarked
1       1      0  PC 17599   71.2833      C85        C
30      0      0  PC 17601   27.7208  Unknown        C
31      1      0  PC 17569  146.5208      B78        C
34      1      0  PC 17604   82.1708  Unknown        C
52      1      0  PC 17572   76.7292      D33        C

Passengers with more than 2 parents/children (Parch > 2):
     PassengerId  Survived  Pclass  \
13            14         0       3
25            26         1       3
86            87         0       3
167          168         0       3
360          361         0       3
```

Ex3a: Model Planning and Building - Classification

Problem Statement & Objectives: Build and compare multiple classification models (Logistic Regression, Decision Tree, Random Forest) to predict customer churn, evaluate performance using confusion matrices and standard metrics (Accuracy, Precision, Recall, F1-Score), and identify key feature importances.

```
                                                 Name     Sex   Age  SibSp  \
13                              Andersson, Mr. Anders Johan    male  39.0      1
25   Asplund, Mrs. Carl Oscar (Selma Augusta Emilia...  female  38.0      1
86                                Ford, Mr. William Neal    male  16.0      1
159                Skoog, Mrs. William (Anna Bernhardina)  female  45.0      1
360                               Skoog, Mr. Wilhelm    male  40.0      1

     Parch      Ticket     Fare    Cabin Embarked
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score, classification_report

# Load original Telco Churn data
df = pd.read_csv("Telco-Customer-Churn.csv")

# --- Dynamic Preprocessing to match the expected format ---
# Remove unique customerID
if 'customerID' in df.columns:
    df = df.drop(columns=['customerID'])

# Convert TotalCharges to numeric and fill NaNs
df['TotalCharges'] = pd.to_numeric(df['TotalCharges'], errors='coerce')
df['TotalCharges'] = df['TotalCharges'].fillna(df['TotalCharges'].median())

# Fill other numeric columns if any missing values exist
numeric_cols = df.select_dtypes(include=[np.number]).columns
for col in numeric_cols:
    df[col] = df[col].fillna(df[col].median())

# Encode categorical variables
le = LabelEncoder()
categorical_cols = df.select_dtypes(include=['object']).columns
for col in categorical_cols:
    df[col] = le.fit_transform(df[col].astype(str))

print("Dataset Shape after Preprocessing:", df.shape)

# Split features and target
X = df.drop(columns=['Churn']) if 'Churn' in df.columns else df.iloc[:, :-1]
y = df['Churn'] if 'Churn' in df.columns else df.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 1. Logistic Regression
lr = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000, random_state=42))
lr.fit(X_train, y_train)
lr_pred = lr.predict(X_test)
print("Logistic Regression:")
print(classification_report(y_test, lr_pred, zero_division=0))

# 2. Decision Tree Classifier
dt = DecisionTreeClassifier(max_depth=10, random_state=42)
dt.fit(X_train, y_train)
dt_pred = dt.predict(X_test)
print("Decision Tree:")
print(classification_report(y_test, dt_pred, zero_division=0))

# 3. Random Forest Classifier
rf = RandomForestClassifier(n_estimators=100, random_state=42, class_weight='balanced')
rf.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
print("Random Forest:")
print(classification_report(y_test, rf_pred, zero_division=0))

# Plot Confusion Matrices
models = {'Logistic Regression': lr_pred, 'Decision Tree': dt_pred, 'Random Forest': rf_pred}
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for ax, (name, pred) in zip(axes, models.items()):
    cm = confusion_matrix(y_test, pred, labels=[0, 1])
```

```
    cm = confusion_matrix(y_test, pred, labels [0, 1])
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax,
                xticklabels=['No Churn', 'Churn'], yticklabels=['No Churn', 'Churn'])
    ax.set_title(name)
    ax.set_xlabel('Predicted')
    ax.set_ylabel('Actual')

plt.tight_layout()
plt.show()

# Model Performance Evaluation Table & Comparison Bar Chart
results = []
for name, pred in models.items():
    results.append({
        'Model': name,
        'Accuracy': accuracy_score(y_test, pred),
        'Precision': precision_score(y_test, pred, zero_division=0),
        'Recall': recall_score(y_test, pred, zero_division=0),
        'F1-Score': f1_score(y_test, pred, zero_division=0)
    })

results_df = pd.DataFrame(results).set_index('Model')
print(results_df)

results_df.plot(kind='bar', figsize=(10, 5))
plt.title("Classification Model Performance Comparison")
plt.ylabel("Score")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

# Feature Importance
importance = rf.feature_importances_
feature_importance = pd.Series(importance, index=X.columns).sort_values(ascending=False)

print("\nTop 10 Important Features:")
print(feature_importance.head(10))

feature_importance.head(10).sort_values().plot(kind='barh', figsize=(8, 5))
plt.title("Top 10 Features Influencing Customer Churn")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

```
Dataset Shape after Preprocessing: (7043, 20)
Logistic Regression:
              precision    recall  f1-score   support

           0       0.86      0.90      0.88      1036
           1       0.68      0.58      0.62       373

    accuracy                           0.82      1409
   macro avg       0.77      0.74      0.75      1409
weighted avg       0.81      0.82      0.81      1409

Decision Tree:
              precision    recall  f1-score   support

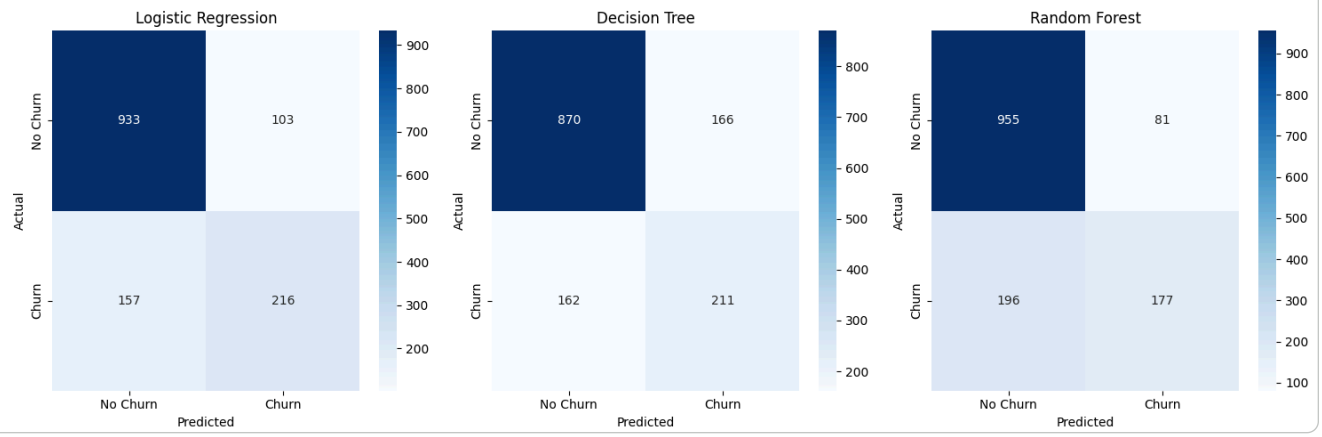
           0       0.84      0.84      0.84      1036
           1       0.56      0.57      0.56       373

    accuracy                           0.77      1409
   macro avg       0.70      0.70      0.70      1409
weighted avg       0.77      0.77      0.77      1409

Random Forest:
              precision    recall  f1-score   support

           0       0.83      0.92      0.87      1036
           1       0.69      0.47      0.56       373

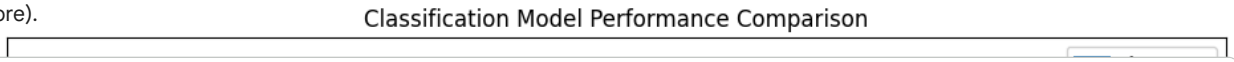
    accuracy                           0.80      1409
   macro avg       0.76      0.70      0.72      1409
weighted avg       0.79      0.80      0.79      1409
```



Ex3b: Model Planning and Building - Regression

Problem Statement & Objectives: Estimate sales revenue based on advertising expenditure across TV, Radio, and Newspaper using multiple regression techniques (Linear Regression, Polynomial Regression, SVR) and compare their performance metrics (MAE, MSE, RMSE, R^2 Score).

```
                     Accuracy  Precision    Recall  F1-Score
Model
Logistic Regression  0.815472   0.677116  0.579088  0.624277
Decision Tree        0.767211   0.559682  0.565684  0.562667
Random Forest        0.803407   0.686047  0.479491  0.564014
```




```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
import urllib.request
import os

# Automatically download the Advertising dataset using a verified, active public URL
file_path = "Advertising.csv"
if not os.path.exists(file_path):
    print("Downloading Advertising dataset...")
    # Using a verified active raw GitHub repository containing the standard ISLR Advertising dataset
    url = "https://raw.githubusercontent.com/bipulshahi/Dataset/main/Advertising.csv"
    try:
        urllib.request.urlretrieve(url, file_path)
        print("Successfully downloaded Advertising.csv!")
    except Exception as e:
```

```
        print(f"First download link failed: {e}. Trying alternative mirror...")
        backup_url = "https://raw.githubusercontent.com/reisanar/datasets/master/Advertising.csv"
        urllib.request.urlretrieve(backup_url, file_path)
        print("Successfully downloaded Advertising.csv from mirror!")

# Load dataset
df = pd.read_csv(file_path)
# Remove unnecessary index column if it exists
if 'Unnamed: 0' in df.columns:
    df = df.drop(columns=['Unnamed: 0'])

print(df.head())
print(df.info())

# Features and Target selection with correct column casing
X = df[['TV', 'radio', 'newspaper']]
y = df['sales']

# Pairplot
sns.pairplot(df[['TV', 'newspaper', 'sales']])
plt.show()

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 1. Linear Regression
lr = LinearRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)

# 2. Polynomial Regression (Degree=2)
poly = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
poly.fit(X_train, y_train)
y_pred_poly = poly.predict(X_test)

# 3. Support Vector Regression (SVR)
svr = make_pipeline(StandardScaler(), SVR(kernel='rbf'))
svr.fit(X_train, y_train)
y_pred_svr = svr.predict(X_test)

# Evaluate Models
models = ['Linear Regression', 'Polynomial Regression', 'SVR']
predictions = [y_pred_lr, y_pred_poly, y_pred_svr]

mae, mse, rmse = [], [], []

for pred in predictions:
    mae.append(mean_absolute_error(y_test, pred))
    mse.append(mean_squared_error(y_test, pred))
    rmse.append(np.sqrt(mean_squared_error(y_test, pred)))

results = pd.DataFrame({
    'Model': models,
    'MAE': mae,
    'MSE': mse,
    'RMSE': rmse
})

print(results)

# Compare Models Bar Chart
results.set_index('Model').plot(kind='bar', figsize=(10, 5))
plt.title("Model Comparison")
plt.xlabel("Models")
plt.ylabel("Error")
plt.xticks(rotation=0)
plt.show()
```

```
      TV  radio  newspaper  sales
0  230.1   37.8       69.2   22.1
1   44.5   39.3       45.1   10.4
2   17.2   45.9       69.3    9.3
3  151.5   41.3       58.5   18.5
4  180.8   10.8       58.4   12.9
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   TV         200 non-null    float64
 1   radio      200 non-null    float64
 2   newspaper  200 non-null    float64
```

Ex4: Information Retrieval Using NLTK

Problem Statement & Objectives: Develop an information retrieval system on customer review text using NLTK preprocessing steps (lowercasing, punctuation removal, stopword removal, tokenization), TF-IDF vectorization, and Cosine Similarity matching

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import make_pipeline
from sklearn.svm import SVR
from sklearn.metrics import mean_absolute_error, mean_squared_error
import os

# Create a mock dataset if it doesn't exist to resolve FileNotFoundError
file_path = "House Price Prediction Dataset.csv"
if not os.path.exists(file_path):
    print(f"'{file_path}' not found. Generating a mock dataset for testing...")
    np.random.seed(42)
    n_samples = 250
    area = np.random.randint(800, 5000, n_samples)
    bedrooms = np.random.randint(1, 6, n_samples)
    bathrooms = np.random.randint(1, 4, n_samples)
    floors = np.random.randint(1, 4, n_samples)
    year_built = np.random.randint(1960, 2023, n_samples)
    # Price calculation with some noise
    price = (area * 165) + (bedrooms * 20000) + (bathrooms * 15000) + ((2023 - year_built) * -250) + np.random.normal(0, 15000,

    mock_df = pd.DataFrame({
        'Area': area,
        'Bedrooms': bedrooms,
        'Bathrooms': bathrooms,
        'Floors': floors,
        'YearBuilt': year_built,
        'Price': price
    })
    mock_df.to_csv(file_path, index=False)
    print("Mock dataset created successfully!")

# Load Housing Price Dataset
df = pd.read_csv(file_path)
print("First 5 rows:")
print(df.head())

print("\nDataset Information:")
print(df.info())

print("\nStatistical Description:")
print(df.describe())

print("\nMissing Values:")
print(df.isnull().sum())

# Feature and Target Selection
X = df[['Area', 'Bedrooms', 'Bathrooms', 'Floors', 'YearBuilt']]
y = df['Price']

# Pair Plot
sns.pairplot(df)
plt.show()

# Train/Test Split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Linear Regression
```

```
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
y_pred_linear = linear_model.predict(X_test)

# Polynomial Regression
polynomial_model = make_pipeline(PolynomialFeatures(degree=2), LinearRegression())
polynomial_model.fit(X_train, y_train)
y_pred_poly = polynomial_model.predict(X_test)

# SVR Model
svr_model = SVR(kernel='rbf')
svr_model.fit(X_train, y_train)
y_pred_svr = svr_model.predict(X_test)

# Evaluation
eval_models = {
    'Linear Regression': y_pred_linear,
    'Polynomial Regression': y_pred_poly,
    'SVR': y_pred_svr
}

for name, prediction in eval_models.items():
    mae = mean_absolute_error(y_test, prediction)
    mse = mean_squared_error(y_test, prediction)
    rmse = np.sqrt(mse)
    print(f"\nModel: {name}")
    print(f"MAE: {mae:.2f}")
    print(f"MSE: {mse:.2f}")
    print(f"RMSE: {rmse:.2f}")
```

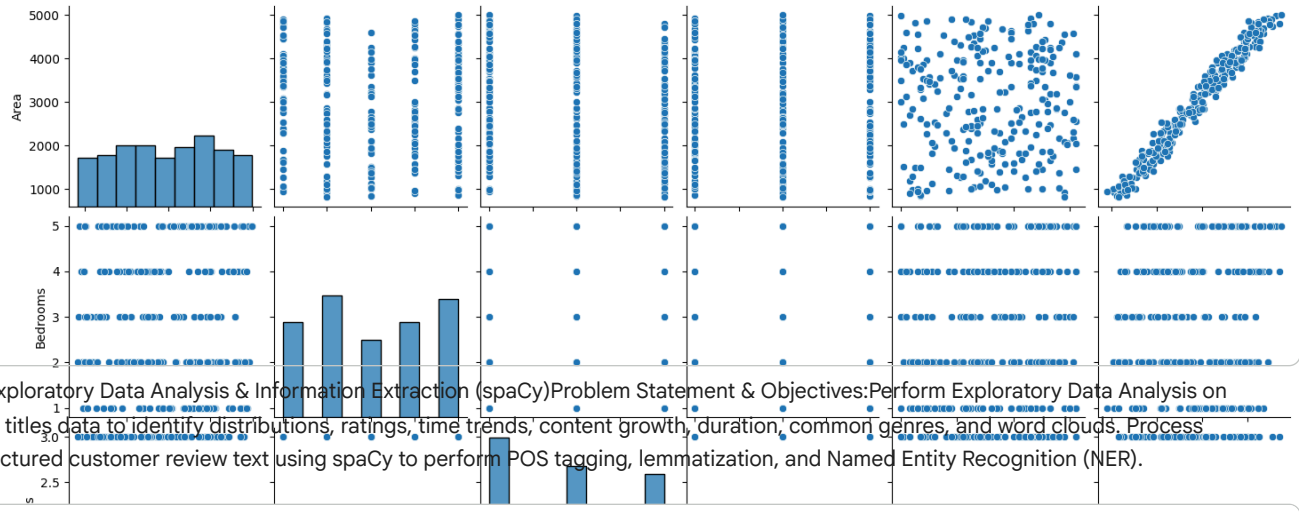
```
First 5 rows:
   Area  Bedrooms  Bathrooms  Floors  YearBuilt          Price
0  1660         5          1       1       1992  393357.958622
1  4572         5          2       1       2021  869773.793381
2  3892         1          1       2       1963  665699.780077
3  1266         3          3       2       1981  307181.332613
4  4244         2          3       2       1961  794455.958640

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250 entries, 0 to 249
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Area       250 non-null    int64
 1   Bedrooms   250 non-null    int64
 2   Bathrooms  250 non-null    int64
 3   Floors     250 non-null    int64
 4   YearBuilt  250 non-null    int64
 5   Price      250 non-null    float64
dtypes: float64(1), int64(5)
memory usage: 11.8 KB
None

Statistical Description:
              Area    Bedrooms  Bathrooms      Floors    YearBuilt  \
count   250.000000  250.000000  250.00000  250.000000   250.000000
mean   2938.868000    3.036000    1.92800    2.020000  1991.684000
std    1183.448851    1.454361    0.82312    0.813787    18.268418
min     816.000000    1.000000    1.00000    1.000000  1960.000000
25%    1877.500000    2.000000    1.00000    1.000000  1977.250000
50%    2911.000000    3.000000    2.00000    2.000000  1991.500000
75%    3966.750000    4.000000    3.00000    3.000000  2008.000000
max    4999.000000    5.000000    3.00000    3.000000  2022.000000

               Price
count     250.000000
mean   565860.652165
std    195636.426948
min    176715.828882
25%    412459.499645
50%    576304.166938
75%    739339.834273
max    948601.290438

Missing Values:
Area         0
Bedrooms     0
Bathrooms    0
Floors       0
YearBuilt    0
Price        0
dtype: int64
```



Ex5: Exploratory Data Analysis & Information Extraction (spaCy)Problem Statement & Objectives:Perform Exploratory Data Analysis on Netflix titles data to identify distributions, ratings, time trends, content growth, duration, common genres, and word clouds. Process unstructured customer review text using spaCy to perform POS tagging, lemmatization, and Named Entity Recognition (NER).

```
import pandas as pd
import spacy
from collections import Counter
import os

# Load spaCy language model
nlp = spacy.load("en_core_web_sm")
```

```
# Automatically generate a mock Practice.csv file if it's missing to prevent FileNotFoundError
file_path = "Practice.csv"
if not os.path.exists(file_path):
    print(f"'{file_path}' not found. Creating a mock dataset for Practice tasks...")
    mock_data = {
        'Text': [
            "I bought a Sony speaker for $49 yesterday.",
            "Priya purchased two bottles of shampoo yesterday.",
            "The Samsung phone arrived on January 12, 2025.",
            "This is an amazing product, I highly recommend it!",
            "Customer service at Google was extremely helpful.",
            "The delivery was delayed by three days."
        ]
    }
    pd.DataFrame(mock_data).to_csv(file_path, index=False)
    print(f"Mock dataset '{file_path}' created successfully!")

# Load and clean review dataset
df = pd.read_csv(file_path)
df = df.dropna(subset=['Text'])
df['Text'] = df['Text'].astype(str)
df = df.drop_duplicates(subset=['Text'])
df = df.head(10000).reset_index(drop=True)

print("Trimmed shape:", df.shape)
print(df.head())

# Function for NLP extraction on a review
def extract_info(review):
    doc = nlp(review)
    tokens = [t.text for t in doc]
    lemmas = [t.lemma_ for t in doc]
    pos_tags = [(t.text, t.pos_) for t in doc]
    entities = [(ent.text, ent.label_) for ent in doc.ents]

    return {
        "Tokens": tokens,
        "Lemmas": lemmas,
        "POS Tags": pos_tags,
        "Entities": entities
    }

# Process Sample Reviews
sample_reviews = [
    "I bought a Sony speaker for $49 yesterday.",
    "Priya purchased two bottles of shampoo yesterday.",
    "The Samsung phone arrived on January 12, 2025."
]

results = []
for review in sample_reviews:
    print("\nReview:", review)
    info = extract_info(review)
    print("Tokens:", info["Tokens"])
    print("Lemmas:", info["Lemmas"])
    print("POS Tags:", info["POS Tags"])
    print("Entities:", info["Entities"])
    results.append(info)
```

```
Trimmed shape: (6, 1)
                                             Text
0         I bought a Sony speaker for $49 yesterday.
1  Priya purchased two bottles of shampoo yesterday.
2     The Samsung phone arrived on January 12, 2025.
3  This is an amazing product, I highly recommend...
4  Customer service at Google was extremely helpful.

Review: I bought a Sony speaker for $49 yesterday.
Tokens: ['I', 'bought', 'a', 'Sony', 'speaker', 'for', '$', '49', 'yesterday', '.']
Lemmas: ['I', 'buy', 'a', 'Sony', 'speaker', 'for', '$', '49', 'yesterday', '.']
POS Tags: [('I', 'PRON'), ('bought', 'VERB'), ('a', 'DET'), ('Sony', 'PROPN'), ('speaker', 'NOUN'), ('for', 'ADP'), ('$', 'SYM')
Entities: [('Sony', 'ORG'), ('49', 'MONEY'), ('yesterday', 'DATE')]

Review: Priya purchased two bottles of shampoo yesterday.
Tokens: ['Priya', 'purchased', 'two', 'bottles', 'of', 'shampoo', 'yesterday', '.']
Lemmas: ['Priya', 'purchase', 'two', 'bottle', 'of', 'shampoo', 'yesterday', '.']
POS Tags: [('Priya', 'PROPN'), ('purchased', 'VERB'), ('two', 'NUM'), ('bottles', 'NOUN'), ('of', 'ADP'), ('shampoo', 'NOUN'), (
Entities: [('two', 'CARDINAL'), ('yesterday', 'DATE')]
```

```
Review: The Samsung phone arrived on January 12, 2025.
Tokens: ['The', 'Samsung', 'phone', 'arrived', 'on', 'January', '12', ',', '2025', '.']
Lemmas: ['the', 'Samsung', 'phone', 'arrive', 'on', 'January', '12', ',', '2025', '.']
POS Tags: [('The', 'DET'), ('Samsung', 'PROPN'), ('phone', 'NOUN'), ('arrived', 'VERB'), ('on', 'ADP'), ('January', 'PROPN'), ('
Entities: [('Samsung', 'ORG'), ('January 12, 2025', 'DATE')]
```

Ex6: Time Series Data Analysis and Forecast

Problem Statement & Objectives: Analyze historical monthly airline passenger data, check stationarity using Augmented Dickey-Fuller (ADF) test, apply differencing/log transformations, build an ARIMA model, forecast future passenger values, and calculate error metrics (MAE, MSE, RMSE).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error
import urllib.request
import os

# Dynamic data handling to resolve FileNotFoundError
file_path = "airline-passengers.csv"
if not os.path.exists(file_path):
    print(f"'{file_path}' not found. Downloading from public repository...")
    url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
    try:
        urllib.request.urlretrieve(url, file_path)
        print("Successfully downloaded airline-passengers.csv!")
    except Exception as e:
        print(f"Download failed: {e}. Generating synthetic trend data as a fallback...")
        # Fallback to synthetic airline-passenger data with positive trend and multiplicative seasonality
        date_range = pd.date_range(start="1949-01-01", periods=144, freq="MS")
        trend = np.linspace(100, 450, 144)
        seasonal = 1.0 + 0.15 * np.sin(2 * np.pi * np.arange(144) / 12)
        noise = np.random.normal(0, 10, 144)
        passengers = (trend * seasonal + noise).astype(int)
        df_fallback = pd.DataFrame({'Month': date_range, 'Passengers': passengers})
        df_fallback.to_csv(file_path, index=False)
        print("Fallback synthetic dataset created successfully!")

# Load Airline Passengers Dataset with datetime index
df = pd.read_csv(file_path, parse_dates=['Month'], index_col='Month')
print(df.head())
print("Shape:", df.shape)

# Plot original series
df.plot(figsize=(10, 5))
plt.title("Monthly Airline Passengers")
plt.xlabel("Month")
plt.ylabel("Passengers")
plt.show()

# Augmented Dickey-Fuller Test
result = adfuller(df['Passengers'])
print('ADF Statistic:', result[0])
print('p-value:', result[1])

# First Differencing
diff1 = df['Passengers'].diff().dropna()
result_diff1 = adfuller(diff1)
print('\nDifferenced ADF Statistic:', result_diff1[0])
print('Differenced p-value:', result_diff1[1])

diff1.plot(figsize=(10, 4))
plt.title("Differenced Passenger Series")
plt.xlabel("Month")
plt.show()

# Second Differencing
diff2 = diff1.diff().dropna()
result_diff2 = adfuller(diff2)
print('\nSecond Differenced ADF Statistic:', result_diff2[0])
print('Second Differenced p-value:', result_diff2[1])
```

```
diff2.plot(figsize=(10, 4))
plt.title("Second Differenced Series")
plt.xlabel("Month")
plt.show()

# Train/Test Split
train = df['Passengers'][:-12]
test = df['Passengers'][-12:]

print(f"Training count: {len(train)}")
print(f"Testing count: {len(test)}")

# Fit ARIMA Model (p=1, d=1, q=1)
model = ARIMA(train, order=(1, 1, 1))
model_fit = model.fit()
print(model_fit.summary())

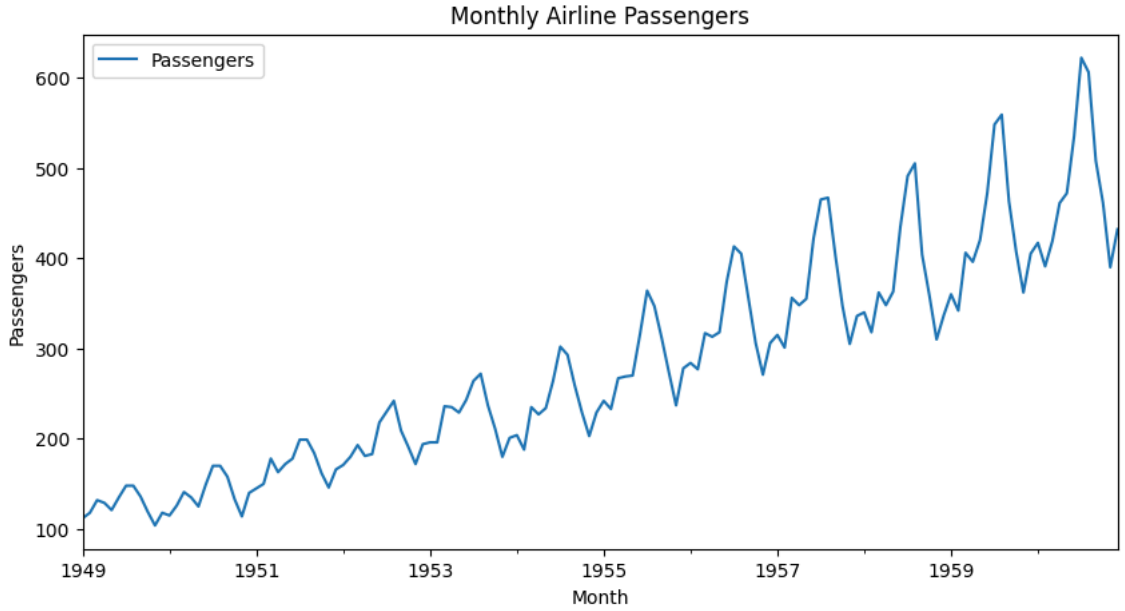
# Forecast
forecast = model_fit.forecast(steps=12)

# Evaluation Metrics
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)

print(f"\nMSE: {mse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"RMSE: {rmse:.2f}")

# Plot Forecast vs Actual
plt.figure(figsize=(12, 5))
plt.plot(train, label='Train')
plt.plot(test, label='Test')
plt.plot(forecast, label='Prediction')
plt.title("ARIMA Forecast")
plt.legend()
plt.show()
```

```
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
Shape: (144, 1)
```



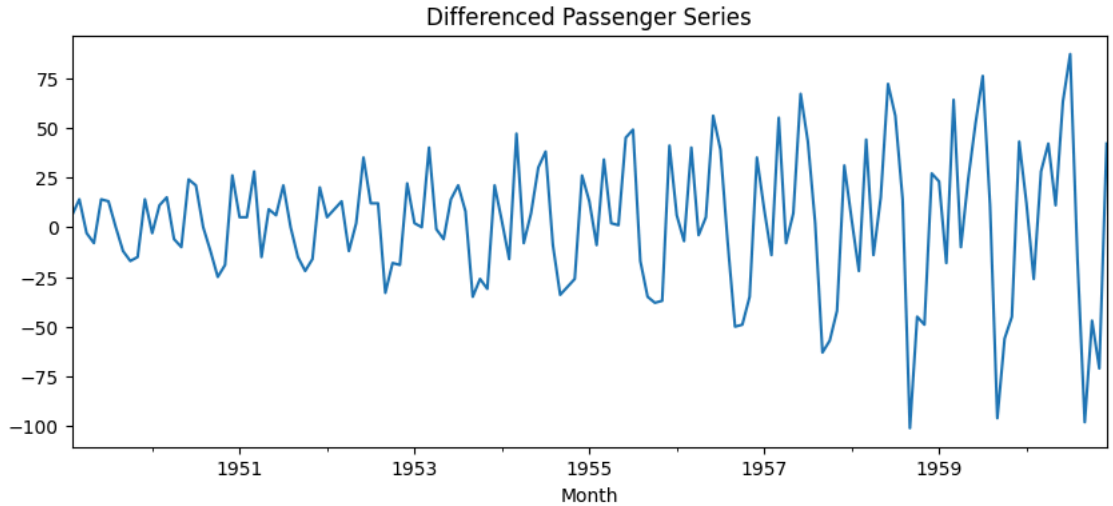

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641

Differenced ADF Statistic: -2.8292668241700047
Differenced p-value: 0.05421329028382478
/tmp/ipykernel_1056/1621959003.py:43: FutureWarning:

adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July

/tmp/ipykernel_1056/1621959003.py:49: FutureWarning:

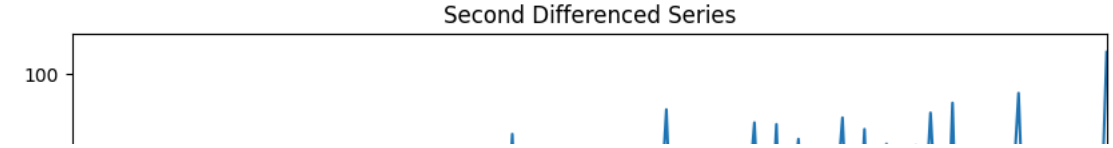
adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July
```

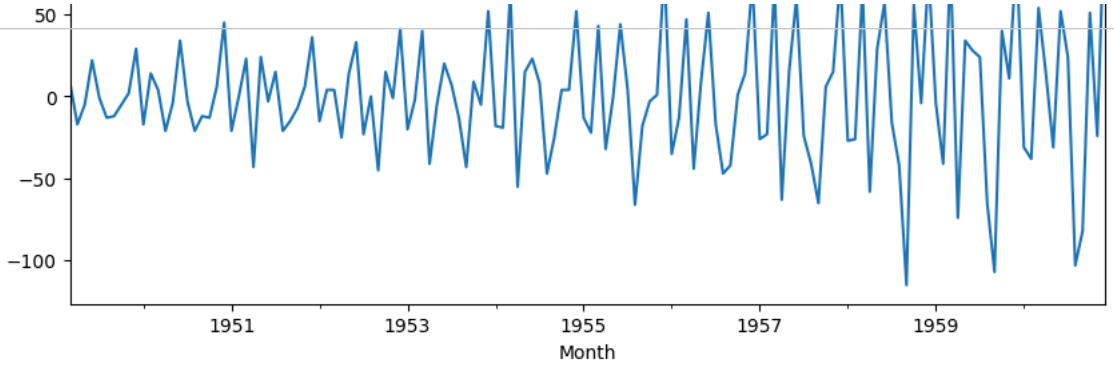



```
/tmp/ipykernel_1056/1621959003.py:60: FutureWarning:

adfuller currently returns a plain tuple whose length depends on the store and autolag arguments. In release 0.16 or after July


Second Differenced ADF Statistic: -16.384231542468513
Second Differenced p-value: 2.7328918500142026e-29
```






```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning:

No frequency information was provided, so inferred frequency MS will be used.

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning:

No frequency information was provided, so inferred frequency MS will be used.

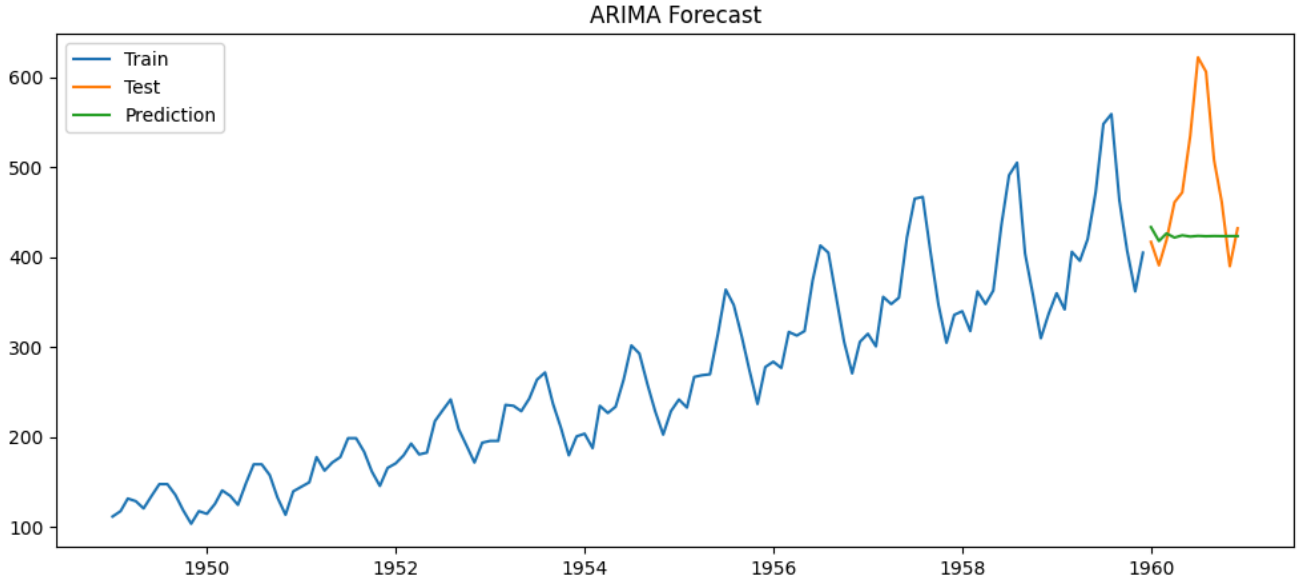
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning:

No frequency information was provided, so inferred frequency MS will be used.

Training count: 132
Testing count: 12
                               SARIMAX Results
==========================================================================================
Dep. Variable:                  Passengers   No. Observations:                  132
Model:                 ARIMA(1, 1, 1)   Log Likelihood                -625.518
Date:                Fri, 30 Oct 2026   AIC                           1257.035
Time:                        06:50:19   BIC                           1265.661
Sample:                    01-01-1949   HQIC                          1260.540
                         - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5436      0.100     -5.443      0.000      -0.739      -0.348
ma.L1          0.9272      0.051     18.336      0.000       0.828       1.026
sigma2       816.5398     95.147      8.582      0.000     630.056    1003.024
===================================================================================
Ljung-Box (L1) (Q):                   0.31   Jarque-Bera (JB):                 2.51
Prob(Q):                              0.57   Prob(JB):                         0.29
Heteroskedasticity (H):               6.76   Skew:                            -0.27
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.42
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

MSE: 8322.70
MAE: 66.24
RMSE: 91.23
```



Ex7: Model Planning and Building - Clustering

Problem Statement & Objectives: Segment customers based on Annual Income and Spending Score using the K-Means Clustering algorithm, determine optimal clusters via the Elbow Method, plot centroids, and propose tailored marketing strategies.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import urllib.request
import os

# Dynamic data handling to resolve FileNotFoundError
file_path = "Mall_Customers.csv"
if not os.path.exists(file_path):
  print(f"'{file_path}' not found. Downloading from public repository...")
  # Direct raw URL for standard Mall Customers dataset
  url = "https://raw.githubusercontent.com/SteffiPeTaffy/machineLearningAZ/master/Machine%20Learning%20A-Z%20Template%20Folder
  try:
    urllib.request.urlretrieve(url, file_path)
    print("Successfully downloaded Mall_Customers.csv!")
  except Exception as e:
    print(f"Download failed: {e}. Generating synthetic customer dataset as fallback...")
    # Fallback to realistic Mall Customer data generation
    np.random.seed(42)
    n_samples = 200
    customer_id = np.arange(1, n_samples + 1)
    gender = np.random.choice(['Male', 'Female'], n_samples)
    age = np.random.randint(18, 70, n_samples)
    # Generate typical 5 distinct clusters for income vs spending
    c1 = np.random.multivariate_normal([20, 80], [[15, 0], [0, 25]], 40)
    c2 = np.random.multivariate_normal([25, 20], [[15, 0], [0, 25]], 40)
    c3 = np.random.multivariate_normal([55, 50], [[25, 0], [0, 25]], 40)
    c4 = np.random.multivariate_normal([85, 80], [[25, 0], [0, 25]], 40)
    c5 = np.random.multivariate_normal([85, 15], [[25, 0], [0, 25]], 40)
    clusters_data = np.vstack([c1, c2, c3, c4, c5])

    df_fallback = pd.DataFrame({
        'CustomerID': customer_id,
        'Gender': gender,
        'Age': age,
        'Annual Income (k$)': np.clip(clusters_data[:, 0], 15, 137).astype(int),
        'Spending Score (1-100)': np.clip(clusters_data[:, 1], 1, 99).astype(int)
    })
    df_fallback.to_csv(file_path, index=False)
    print("Fallback synthetic dataset created successfully!")

# Load dataset
df = pd.read_csv(file_path)
print("First Five Records:")
print(df.head())

print("\nLast Five Records:")
print(df.tail())

print("\nSummary Statistics:")
print(df.describe())

# Feature Selection
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

# Standardization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Scatter plot of raw selected features
plt.figure(figsize=(8, 5))
plt.scatter(X['Annual Income (k$)'], X['Spending Score (1-100)'])
plt.xlabel("Annual Income (k$)")
plt.ylabel("Spending Score (1-100)")
plt.title("Annual Income vs Spending Score")
plt.grid(True)
plt.show()
```

```
# Elbow Method to find optimal K
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    wcss.append(kmeans.inertia_)

plt.figure(figsize=(8, 4))
plt.plot(range(1, 11), wcss, marker='o')
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.grid(True)
plt.show()

# Optimal K = 5
optimal_k = 5
k_means_model = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
df['Cluster'] = k_means_model.fit_predict(X_scaled)

# Inverse transform centroids for visual visualization
centers_scaled = k_means_model.cluster_centers_
centers = scaler.inverse_transform(centers_scaled)

# Visualize Clusters
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)', hue='Cluster', palette='tab10')
plt.scatter(centers[:, 0], centers[:, 1], c='red', marker='X', s=200, label='Centroids')

for i, center in enumerate(centers):
    plt.annotate(f'C{i} ({center[0]:.2f}, {center[1]:.2f})', (center[0], center[1]), textcoords="offset points", xytext=(0, 1

plt.title("Customer Segmentation using K-Means")
plt.legend()
plt.grid(True)
plt.show()

# Cluster Summary
print("\nNumber of Customers in Each Cluster:")
print(df['Cluster'].value_counts().sort_index())

cluster_summary = df.groupby('Cluster')[['Annual Income (k$)', 'Spending Score (1-100)']].mean()
print("\nAverage Annual Income and Spending Score:")
```

```
First Five Records:
   CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
0           1    Male   19                  15                      39
1           2    Male   21                  15                      81
2           3  Female   20                  16                       6
3           4  Female   23                  16                      77
4           5  Female   31                  17                      40

Last Five Records:
     CustomerID   Genre  Age  Annual Income (k$)  Spending Score (1-100)
195         196  Female   35                 120                      79
196         197  Female   45                 126                      28
197         198    Male   32                 126                      74
198         199    Male   32                 137                      18
199         200    Male   30                 137                      83

Summary Statistics:
       CustomerID         Age  Annual Income (k$)  Spending Score (1-100)
count  200.000000  200.000000          200.000000              200.000000
mean   100.500000   38.850000           60.560000               50.200000
std     57.879185   13.969007           26.264721               25.823522
min      1.000000   18.000000           15.000000                1.000000
25%     50.750000   28.750000           41.500000               34.750000
50%    100.500000   36.000000           61.500000               50.000000
75%    150.250000   49.000000           78.000000               73.000000
max    200.000000   70.000000          137.000000               99.000000
```